

"SnackMaster ERP - System Blueprint & Technical Architecture V1.0".

Phase 1: High-Level Foundations

1.1 Project Overview (The "Why")

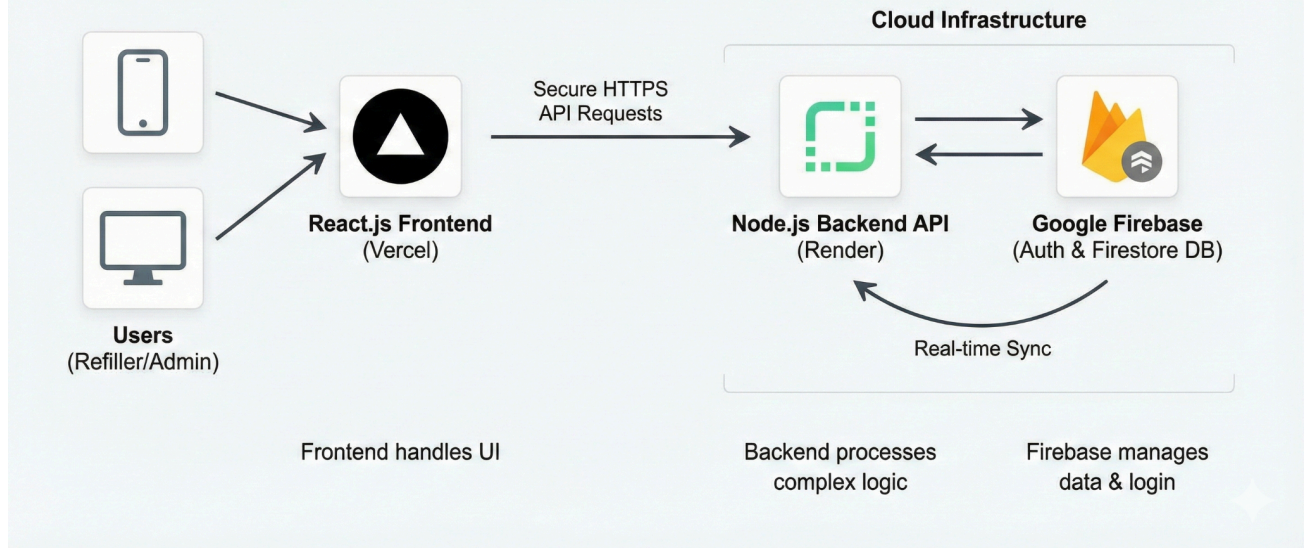
SnackMaster is the operational bridge between vending machine sales, warehouse inventory, and field staff.

- **The Problem:** Vending machines use third-party payment devices that only track money, not physical inventory. Because of this, operators guess what needs refilling, overpack their vehicles, and lose track of warehouse stock.
- **The Solution:** A centralized web platform where Admins upload third-party sales data (CSV). The system instantly reads this data to deduct sold items, update the warehouse ledger, and generate precise "Refill Kits" (shopping lists) for the field staff.
- **Why We Built It This Way:** Instead of building expensive custom hardware to connect directly to the machines, we built a smart software layer that uses existing sales data. It is a highly cost-effective way to solve the business's biggest problem: inventory traceability.
- **Target Users:** Vending machine operators and SaaS businesses managing multiple locations, warehouses, and refill staff.

1.2 System Architecture (The "Big Picture")

This is how the different parts of the project talk to each other. We use a **Modern Cloud Stack**, which means the system is fast, secure, and doesn't require a physical server in an office.

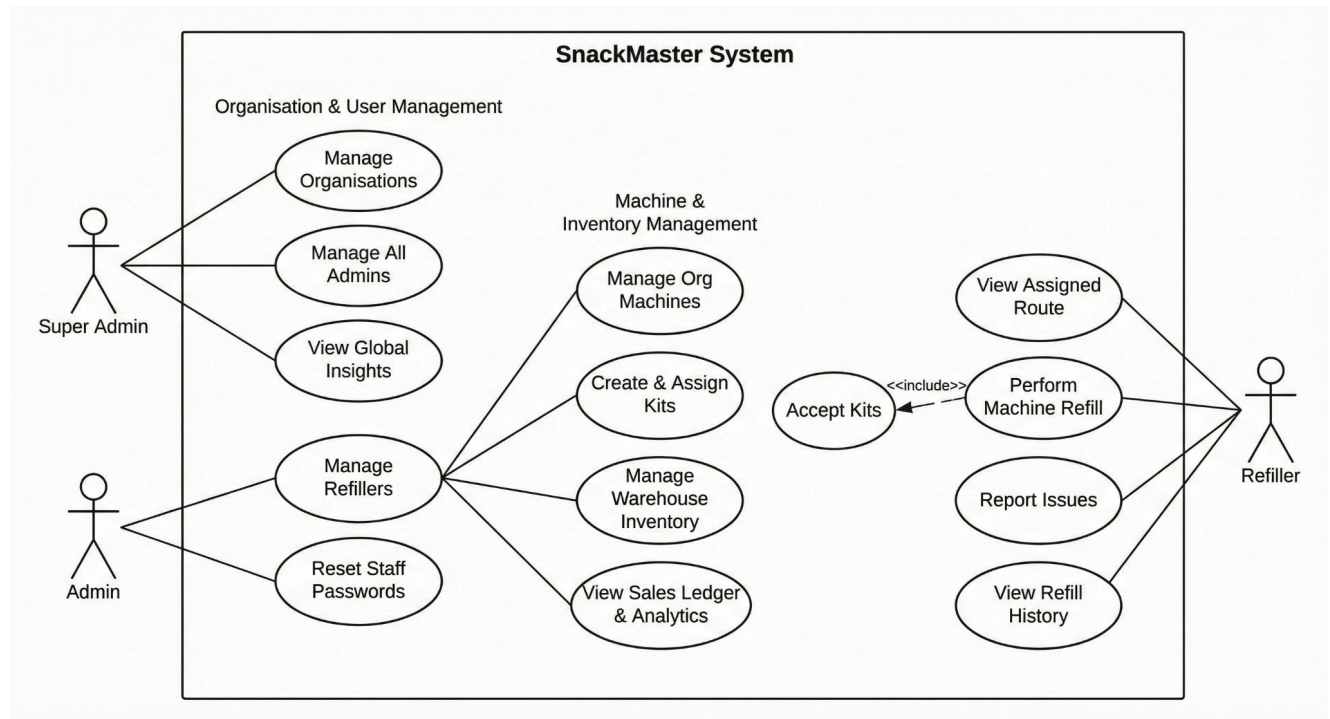
System Architecture Diagram



- **The Frontend (The Face):** Built using **React.js**. This is what users see on their screens. It is optimized for both desktop (Admins) and mobile (Refillers).
- **The Backend (The Engine):** Built using **Node.js** and **Express**. It handles the heavy lifting, like calculating warehouse movements and processing secure password resets.
- **The Database (The Memory):** We use **Google Firebase (Firestore)**. It is a "NoSQL" database, meaning it can store and sync data across all users instantly without lag.
- **Security Layer:** All communication is encrypted using SSL (HTTPS), and the code is "Minified" (scrambled) so it cannot be read by outsiders.

1.3 User Roles & Permissions

To keep the data safe, not everyone can see everything. We use **Role-Based Access Control (RBAC)**.



1. **Super Admin:** The "God Mode." Can create new organizations, manage all machines across the country, and view global insights.
2. **Admin:** The "Hub Manager." Manages a specific organization's machines, products, and warehouse. They can see sales reports but cannot see other companies' data.
3. **Refiller:** The "Field Worker." Has a simplified mobile view. They can only see the machines assigned to them, record stock updates, and report technical issues.

Phase 2: Functional & Technical Flow

This section breaks down the daily operations of the system. It explains the journey of a user and how the data travels securely through the cloud.

2.1 The User Journey (Functional Flow)

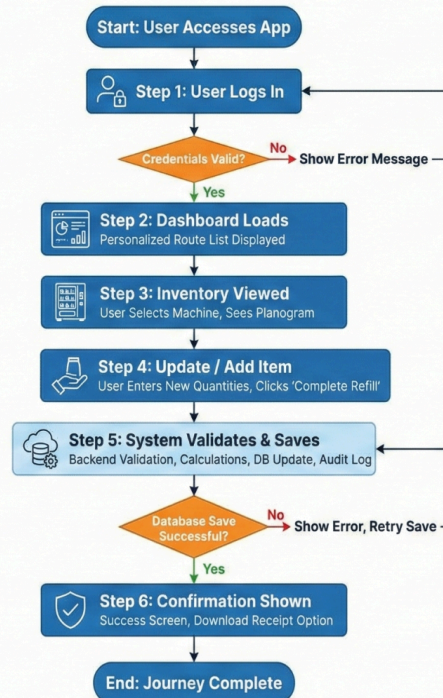
This explains step-by-step what happens when a human interacts with the software.

A. The Refiller's Journey (Field Operations)

Goal: To update a machine's stock level accurately.

1. **Login:** The Refiller opens the app on their mobile browser and logs in with their credentials (e.g., `refill-test@snackmaster.com`).
2. **Dashboard View:** Instead of scanning a code, they see a simple **"My Route" list**. This list shows only the machines assigned to them (e.g., "Gym Lobby", "Office Floor 2").
3. **Selection:** They tap on a machine name. The app shows the current "Planogram" (the map of slots inside that machine).
4. **Audit & Refill:**
 - They look at **Slot 11**.
 - The app asks: "How many Chips are here?"
 - They count and type the number (e.g., "3").
 - They repeat this for all slots.
5. **Submission:** They click **"Complete Refill."**
6. **Instant Sync:** The app sends this data to the cloud. The machine's status immediately changes from "Refill in Progress" to "Active" on the Admin's screen.

SnackMaster - Refiller User Journey (Functional Flow)



B. The Admin's Journey (Hub Management)

Goal: To manage warehouse stock and oversee refills.

1. **Dashboard:** The Admin logs in on a desktop. They see a high-level view: "5 Machines Low Stock," "200 Items Sold Today."
 2. **Kit Creation:**
 - Before a Refiller leaves, the Admin uses the **"Make Kit"** feature.
 - The system calculates exactly what is missing from a machine.
 - It generates a "Shopping List" (Manifest) for the Refiller to take from the warehouse.
 3. **Warehouse Deduction:** When the Refiller accepts the Kit, the system *automatically* subtracts those items from the **Warehouse Inventory**. This ensures the digital inventory always matches the physical warehouse.
 4. **Financial Review:** The Admin checks the **Sales Ledger** to see every transaction and uses the "Trash Bin" to restore any sales data deleted by mistake.
-

2.2 Data Flow Diagram (DFD) - Level 0

This explains how information moves without using technical jargon.

Think of the system like a restaurant kitchen:

1. **The Waiter (Frontend/React):** This is what the user touches. It takes the order (inputs data) and shows the menu (displays data). It doesn't cook the food; it just sends the request.
2. **The Kitchen (Backend/Node.js):** This is on **Render**. It receives the order. It checks if the ingredients exist (Validation) and performs the heavy work (Calculations).
3. **The Pantry (Database/Firebase):** This is where all the raw ingredients (Data) are stored. The Kitchen takes items from here and puts finished dishes back.

The Flow:

1. User clicks "Save" → **Frontend** sends a secure message (API Request).
 2. **Backend** receives the message → Checks "Is this person allowed to do this?" (Security).
 3. **Backend** talks to **Database** → "Update Machine #5 stock to 10."
 4. **Database** confirms "Done."
 5. **Backend** tells **Frontend** "Success!" → User sees a green checkmark.
-

2.3 Database Structure (Simplified)

This describes how we organize data without revealing the exact code structure.

We use a **NoSQL Database** (Firestore). Imagine a giant filing cabinet where every folder is a "Collection" and every paper inside is a "Document."

- 📁 **Users Collection:**
 - Stores profile info (Email, Role, Organization ID).
 - *Security Note:* Passwords are NOT stored here; they are handled separately by a secure Google Authentication vault.
- 📁 **Machines Collection:**
 - Each document represents one physical machine.
 - Contains: Name, Location, Current Stock Level, Last Refill Time.
 - *Sub-Folder: Slots* (Holds details for A1, A2, B1, etc.).
- 📁 **Products Collection:**
 - The master list of items (e.g., "Coke Can," "Lays Chips").
 - Stores: Price, Warehouse Stock Count, Image URL.
- 📁 **Sales Transactions Collection:**
 - The financial ledger.

- Stores: Transaction ID, Amount, Payment Method, Time.
 - 📁 **Logs & Audits:**
 - A hidden security camera. It records *who* did *what* and *when* (e.g., "Admin updated price of Coke at 10:00 AM").
-

2.4 Why this Architecture is Secure (Anti-Replication)

To protect your business value, we designed the system so that the "Brain" is separated from the "Face."

- **Logic Isolation:** The complex math (like calculating exactly how much stock to deduct for a Kit) happens on the **Backend Server**, not on the user's phone. Even if someone inspects the website code, they can't see *how* the calculation is done.
- **Role Barriers:** A Refiller literally *cannot* download the Admin's data because the **Backend** checks their ID badge (Role) before sending any sensitive information.

Phase 3: Security Architecture & API Gateway

3.1 Data Security & Access Control (The "Bouncer")

To ensure data cannot be leaked or accidentally destroyed, we use **Cloud Firestore Security Rules**. Think of this as a digital bouncer at the door of the database.

Every time someone tries to read or write data, the bouncer checks their ID:

- **Authentication Check:** Is this user logged in? (If no, reject).
- **Role Check:** Is this user a Refiller or an Admin?
- **Organization Check:** Does this Admin belong to "Company A"? (If yes, they cannot see "Company B" data).
- **UAT Safety Locks:** For the current testing phase, we have placed a strict **"No-Delete"** lock on critical business data (Machines, Products, Users). Users can create and edit, but cannot permanently delete these items, preventing accidental damage during testing.

3.2 Frontend Protection (Anti-Theft Shield)

If a competitor or a curious user tries to "Inspect" the website code, they will not be able to steal the SnackMaster logic.

- **Code Minification:** We use a modern build tool (Vite + Terser) that scrambles the code into an unreadable format before it is sent to the internet.
- **Source-Map Blocking:** The original "blueprints" of the code are completely hidden from the live browser.
- **Console Privacy:** All developer logs and background tracking texts are automatically stripped out of the live version so sensitive IDs are never exposed.

3.3 API Gateway (The Backend Services)

While most of the system reads data directly and securely from the cloud, certain high-level tasks require a secure, centralized server. Our **Render Node.js Server** handles these sensitive operations.

Here are the primary API endpoints configured for V1.0:

API 1: Machine Refill & Inventory Sync

- **Endpoint:** `POST /api/confirm-refill`
- **Purpose:** Acts as the central brain when a Refiller finishes packing a machine.
- **How it works:**
 1. Receives the final stock counts from the Refiller's phone.
 2. Updates the machine's "Current Stock" to 100%.
 3. Generates a permanent **Refill Audit Log** (who refilled it, what time, what was the variance).
 4. Automatically calculates any "Expired" or "Returned" items and logs them into the **Warehouse Ledger**.
 5. Deducts returned stock back into the physical warehouse inventory.
- **Security:** This uses an "Atomic Batch." This means if the internet drops halfway through, the system cancels the *entire* operation rather than saving half-finished data.

API 2: Admin Password Recovery

- **Endpoint:** `POST /api/admin-reset-password`
- **Purpose:** Allows Admins to securely reset forgotten passwords for their staff.
- **How it works:**
 1. Receives the request from the Admin dashboard.
 2. Bypasses the standard email-link process using secure **Firebase Admin Privileges**.
 3. Forces the target user's password to the default (`Welcome@123`).
 4. Creates an immutable **Audit Trail** noting exactly which Admin triggered the reset.

3.4 Infrastructure & Cloud Secrets

- **No Hardcoded Keys:** Master database passwords and secret keys are **never** stored in the source code.
- **Environment Variables:** Secrets are securely injected directly into the Render and Vercel cloud servers via encrypted Environment Variables. This ensures that even if the GitHub repository were somehow accessed, the hacker would still have no keys to enter the database.

Phase 4: Handover, Deployment & Testing

4.1 Deployment & Hosting Environment

SnackMaster utilizes a modern, serverless hosting architecture. This means there are no physical servers to maintain, and the system scales automatically based on user traffic.

- **Frontend (User Interface): Hosted on Vercel**
 - Vercel serves the React application globally via a CDN (Content Delivery Network), ensuring lightning-fast load times for Admins and Refillers.
 - **CI/CD Pipeline:** The system is connected to a secure, private GitHub repository. Any approved updates pushed to the "main" branch automatically rebuild and deploy to the live site within minutes.
- **Backend (API Server): Hosted on Render**
 - The Node.js server handles heavy data processing (like inventory deductions and password resets). It runs on a secure Render instance.
- **Database & Auth: Google Firebase**
 - Manages user logins, secure password hashing, and real-time database syncing across all devices.

4.2 User Acceptance Testing (UAT) Scenarios

To verify the system is functioning correctly, we recommend the client's testing team execute the following core scenarios:

Test ID	Role	Action to Perform	Expected Result
TC-01	Any	Attempt to log in with an incorrect password.	System displays an "Invalid email or password" error.
TC-02	Admin	Go to Warehouse > Create a "Kit" for a specific machine.	Kit is generated, and machine status changes to "Pending Acceptance".
TC-03	Refiller	Open the assigned machine and click "Accept Kit".	Kit is accepted, and stock is automatically deducted from the Admin's Warehouse Ledger.
TC-04	Refiller	Input final stock counts in the slots and click "Complete Refill".	Machine is marked "Active", current stock updates to 100%, and an Audit Log is generated.
TC-05	Admin	Navigate to Sales Ledger and permanently delete a test transaction.	Transaction moves to the "Trash Bin" for recovery or permanent removal.
TC-06	Admin	Try to delete a Machine or Product from the database.	Action is blocked (UAT Safety Lock is actively preventing data loss).

4.3 Known Boundaries & UAT Configurations

During this V1.0 testing phase, please note the following system configurations:

- **UAT Deletion Lock:** Hard-deletion of core infrastructure (Machines, Products, Organizations) is currently disabled to prevent accidental data loss during familiarization.
- **Hardware Abstraction:** Direct hardware MDB/telemetry syncing is currently handled via the manual/CSV upload logic to the Sales Ledger, keeping the web platform hardware-agnostic.
- **Offline Caching:** If a Refiller temporarily loses internet access (e.g., in a basement), the app will cache data locally and sync automatically once the connection is restored (requires the browser window to remain open).

4.4 Handover Checklist

Upon final sign-off, the following assets will be officially transferred to the organization:

- [x] **Live Application URLs:** (Frontend dashboard and Backend API routes).
- [x] **Testing Credentials:** Secure login details for Super Admin, Admin, and Refiller testing.
- [x] **System Blueprint:** This technical documentation and architecture overview.
- [x] **Source Code Repository:** Access to the private GitHub repository containing the minified and secured source code.

Final Note & Handover Statement

SnackMaster Version 1.0 has been successfully built, tested, and updated based on the initial UAT feedback. The system is now fully equipped to handle standard daily operations, including CSV sales ingestion, warehouse traceability, and field staff routing.

Important System Guidelines:

- **Hosting Environment:** The current application is running on a free-tier hosting environment. It is optimized for the current set of features.
- **Scalability:** To maintain smooth page load speeds and database performance, it is highly recommended to avoid adding heavy new features until the system is migrated to a dedicated, paid production server.
- **Operational Readiness:** The platform is stable and ready for active operational use by your Hub Admins and Refillers.

It has been a pleasure building this operational foundation for the team. Please refer to the specific sections of this document for any workflow queries.

— **Vicky Kumar Singh** *Lead Developer & Software Engineer February 2026*

